Predicting Insurance Claim Severity

Using Machine Learning

DATA 6545: Data Science and MLOps · Spring 2026 · Connor Hernon

LightGBM · SHAP · Flask API · Docker · MLflow · Render

The Problem

High-severity claims that go undetected at intake force insurers into reactive, costly responses

The Challenge

When extreme claims aren't flagged at intake, insurers respond reactively – after costs have already accumulated:

- Inaccurate loss reserves
- Misallocated investigative resources
- No chance for early intervention

The Solution

A two-component ML system built on the Allstate Claims Severity dataset:

- Regression model – estimates dollar loss at claim intake for reserve planning
- Classifier – flags top 10% highest-cost claims for elevated review before costs develop

188,318	130	$3,037	0
Training claims	Features	Avg loss	Missing values

The Data

188,318 training claims · 130 anonymized features · log(1+loss) reduces skewness from 3.79 → 0.09

116 Categorical Features (cat1–cat116)

Mostly binary or low-cardinality. 11 columns have unseen test categories – handled by fitting encoders on combined train+test data.

14 Continuous Features (cont1–cont14)

Pre-scaled to [0,1]. Strong pairwise correlations exist (cont11/cont12 = 0.994) – irrelevant for tree models. Highest single-feature correlation with loss is cont2 at 0.142. Severity is driven by interactions, not individual features.

Statistic	Value
Mean	$3,037
Median	$2,115
Std Dev	$2,904
90th percentile	$6,401.74
Maximum	$121,012
Skewness (raw / after log)	3.79 → 0.09

After log(1+loss), the distribution becomes nearly symmetric – stabilizing model training on this heavy-tailed dataset.

Model Progression

LINEAR BASELINE

Ridge Regression

L2 regularization handles multicollinearity across 130 features. One-hot encoding. Establishes the performance floor — the target a nonlinear model must beat.

MAE $1,244.68 · R^2 0.4149

NONLINEAR BENCHMARK

Random Forest

Confirms the dataset rewards complexity. Captures interactions Ridge misses. Also revealed that one-hot encoding creates unnecessary compute overhead — motivating label encoding for the final model.

MAE $1,223.44 · R^2 0.4823

PRODUCTION MODEL

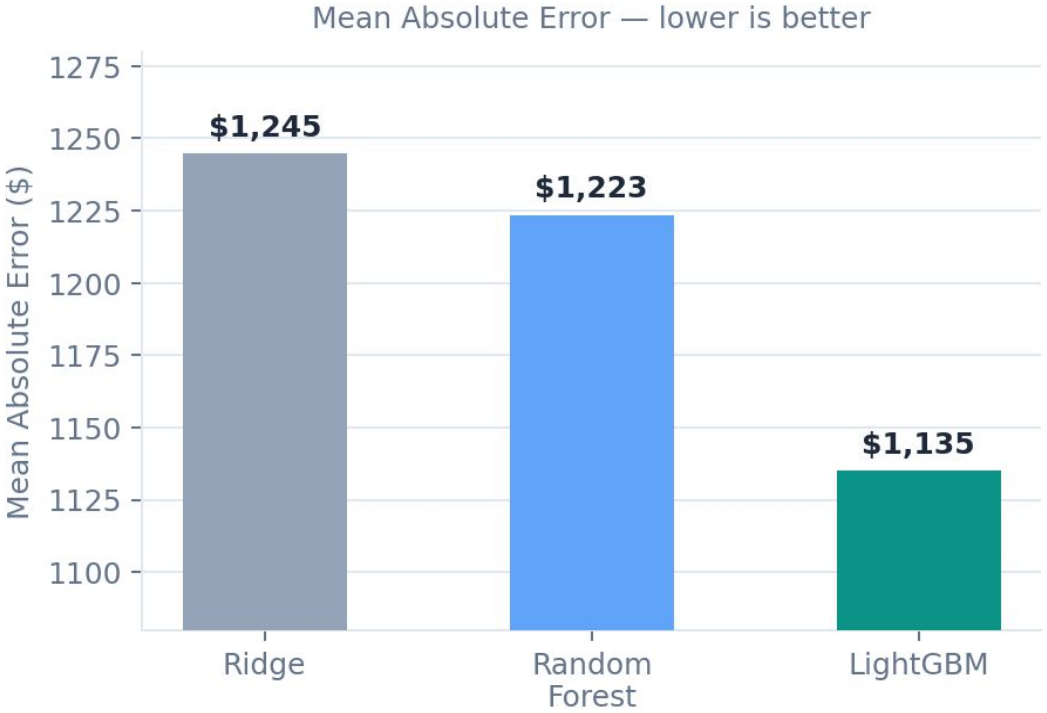
LightGBM

Label encoding removes the sparse matrix bottleneck. Leaf-wise growth + histogram-based splitting. Early stopping (patience 50) on validation MAE prevents overfitting. Fastest and most accurate.

MAE $1,135.16 · R^2 0.5595

LightGBM Wins Across Every Metric

$109 MAE reduction · 125,546 test claims · ~$13.7M in more accurate annual reserves



Model	MAE	RMSE	R^2
Ridge	$1,244.68	$2,185.06	0.4149
Random Forest	$1,223.44	$2,055.23	0.4823
LightGBM	**$1,135.16**	**$1,895.83**	**0.5595**

The key insight

A $109 MAE improvement across 125,546 test claims translates to ~$13.7M in more accurate annual reserves — before any triage benefit from the classifier.

Cross-Validation Confirms Stability

CV of 0.69% — this result is not a lucky split

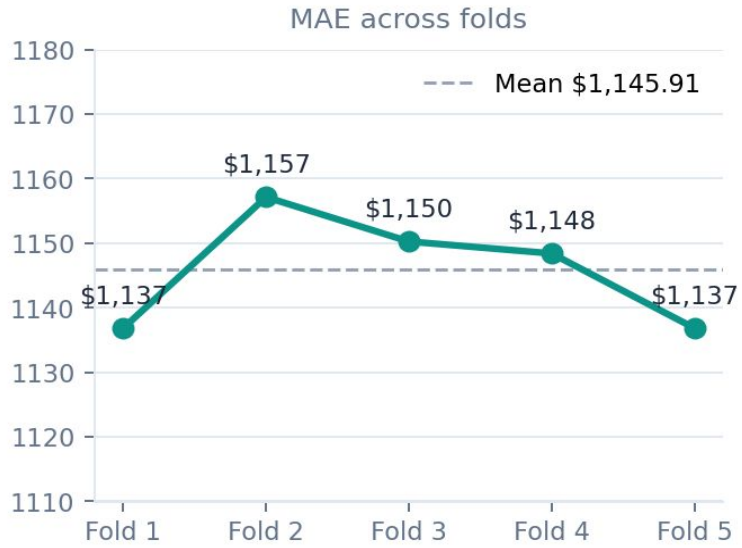
$1,145.91
Mean CV MAE

± $7.95
Std Deviation

0.5519
Mean CV R^2

< 25s
Per-fold Runtime

Fold	MAE	R^2	Time
Fold 1	$1,136.85	0.5589	19.6s
Fold 2	$1,157.15	0.5300	18.4s
Fold 3	$1,150.27	0.5611	22.1s
Fold 4	$1,148.42	0.5663	18.3s
Fold 5	$1,136.85	0.5435	19.1s
Mean	**$1,145.91**	**0.5519**	—



Severity Classifier: AUC 0.9315

Youden's J threshold (0.0746) deployed — recall 0.88, catching nearly 9 in 10 extreme claims

0.9315	0.88	0.36	0.51
AUC-ROC	Recall	Precision	F1 Score

All metrics at Youden's J optimized threshold (0.0746) · High-severity threshold: $6,401.74 (90th percentile of training loss)

Youden's J Threshold (0.0746)

Youden's J maximizes sensitivity + specificity simultaneously. At threshold 0.0746, recall jumps to 0.88 — the model catches nearly 9 in 10 extreme claims. Precision is 0.36, meaning more false flags, but in a triage context catching extreme claims is the priority.

This threshold is fixed in the deployed API and Streamlit dashboard.

The Precision–Recall Tradeoff

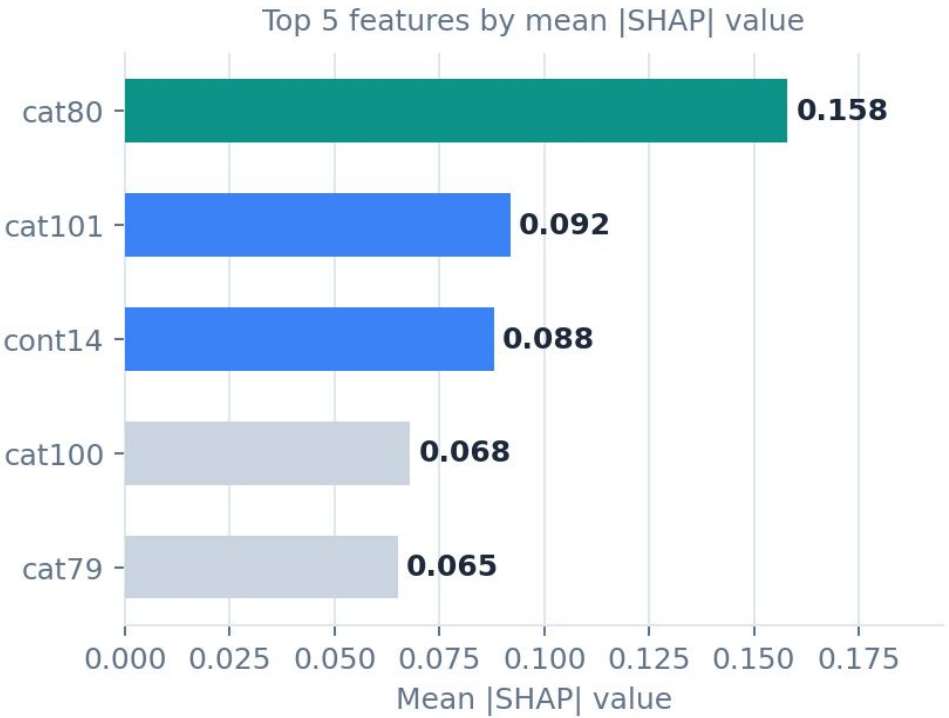
The threshold is a dial, not a fixed number:

- Lower threshold → higher recall, more claims reviewed (current deployment at 0.0746)
- Higher threshold → higher precision, fewer flags but more missed extreme claims

The right setting depends on the team's capacity to absorb false positives vs. the cost of missing a $9,500+ claim.

Why the Model Predicts What It Does

cat80 dominates by 60% margin · cont14's nonlinearity is exactly why Ridge can't compete



cat80 — the dominant driver

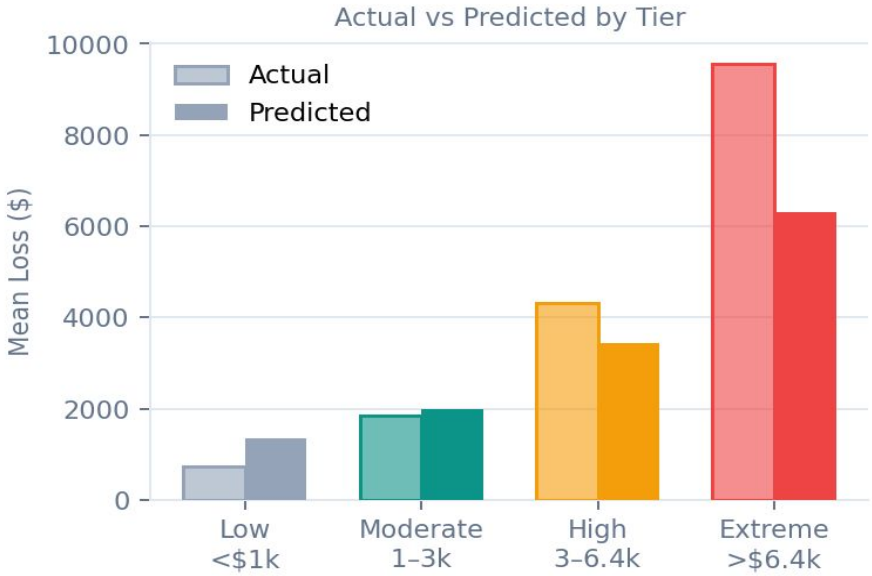
Four encoded levels: 0, 1, 2 push predicted loss upward; level 3 consistently reduces it. This clean four-way segmentation gives cat80 ~60% more predictive weight than the second-ranked feature.

cont14 — the nonlinearity gap

SHAP dependence plot shows sharp jumps at ~0.25 and ~0.82 — a step-function pattern invisible to any linear model regardless of regularization. This single feature largely explains the $109 MAE gap between Ridge and LightGBM.

Error Analysis: Where the System Delivers

Strong performance on the claims that make up the bulk of the portfolio — classifier closes the gap on extreme claims



Moderate tier — best performance
17,830 claims, 36% error. This is 47% of the validation set and the tier closest to the training mean. The model is well-calibrated here.

Low tier — overestimate is minor

103% error sounds alarming. But that's $628 on a $722 claim — operationally inconsequential in dollar terms.

Extreme tier — classifier is the answer
$3,237 mean underprediction is why we built the classifier. At Youden's J threshold, it catches 88% of extreme claims for elevated review before costs accumulate.

End-to-End Production System

All endpoints live · auto-deploys on push · reproducible from a single command

Artifacts	Flask API	Container	Deployment
lgbm_regressor.joblib	GET /health	Docker	Render (live)
lgbm_classifier.joblib	GET /model_info	python:3.11-slim	Auto-deploy on push
label_encoders.joblib	POST /predict	Gunicorn 2 workers	Health check /30s
metadata.joblib	POST /predict_batch	Port 8080	CI/CD via GitHub

MLflow Tracking

All 4 runs logged with hyperparameters, metrics, and CV results. mlflow_run_summary.csv committed to repo for auditability.

Streamlit Dashboard

125,546 test predictions. Filter by severity tier, loss range, flag. Search by claim ID. Live at allstate-claims-dashboard.streamlit.app

Reproducibility

Clone repo → pip install -r requirements.txt → run notebook. Both models retrain end-to-end from the raw Kaggle CSV.

Live Demo: Claims Dashboard

allstate-claims-dashboard.streamlit.app · all 125,546 test predictions · github.com/hernon33/allstate-claims-dashboard

What You'll See

5 summary cards
Total claims, avg predicted loss, % high severity, extreme count, active filter count

Severity tier bar chart

Distribution across Low / Moderate / High / Extreme

Loss histogram

Predicted loss across 8 dollar ranges

Classifier probability histogram
Full distribution with AUC-ROC and Youden's J threshold annotated

Paginated claims table
25 per page, color-coded by tier, sortable, filterable by tier/flag/loss range

Data Source

All 125,546 predictions generated by running the serialized LightGBM regressor and classifier against the encoded test set — identical artifacts to those in the Flask API. Predictions saved as test_predictions_full.csv and committed to the dashboard repo.

Consistency Across the System

The same $6,401.74 threshold and Youden's J value of 0.0746 are used:

- In the training notebook (classifier evaluation)
- In the Flask API (/predict and /predict_batch endpoints)
- In the Streamlit dashboard (flags and tier assignments)

A Complete ML Lifecycle

From raw claims to live API

LightGBM — MAE $1,135.16 · RMSE $1,895.83 · R^2 0.5595 · CV σ $7.95

Severity classifier — AUC 0.9315 · Recall 0.88 at Youden's J threshold (0.0746)

cat80 is the dominant feature · cont14's nonlinearity explains the Ridge–LightGBM MAE gap

Flask API on Render · Docker · MLflow · Streamlit dashboard · GitHub CI/CD

github.com/hernon33/allstate-severity-api · allstate-claims-dashboard.streamlit.app · github.com/hernon33/allstate-claims-dashboard